

FactorEngine: A Program-level Knowledge-Infused Factor Mining Framework for Quantitative Investment

Qinhong Lin¹, Ruitao Feng², Yinglun Feng¹, Zhenxin Huang³,
Yukun Chen¹, Zhongliang Yang^{1,4}, Linna Zhou¹, Binjie Fei², Jiaqi Liu², Yu Li^{2,*}

¹Beijing University of Posts and Telecommunications, China

²Beijing Value Simplex Technology Co., Ltd., China

³Yangtze Delta Research Institute, University of Electronic Science and Technology of China, China

⁴Wuhan Eonray Technology Co., Ltd., China

greenred99@bupt.edu.cn {fengruitao, liyu}@entropyreduce.com

Abstract—We study alpha factor mining—the automated discovery of predictive signals from non-stationary market data, under a practical requirement that mined factors be directly executable and auditable, and that the discovery process remain computationally tractable at scale. Existing symbolic approaches search over fixed operator grammars, while neural forecasters often trade interpretability for performance and remain vulnerable to regime shifts. We introduce FactorEngine (FE), a program-level factor discovery framework that casts factors as execution-constrained code and improves both effectiveness and efficiency via three separations: (i) logic revision vs. market-conditioned instantiation, (ii) LLM-guided directional search vs. Bayesian local search, and (iii) LLM usage vs. local computation. FE further incorporates a knowledge-infused bootstrapping module that transforms unstructured financial reports into executable factor programs through a closed-loop multi-agent extraction–verification–code-generation pipeline, and a trajectory-aware refinement mechanism guided by accumulated evolution experience. Across extensive backtests on real-world OHLCV data, FE produces factors with substantially stronger predictive and portfolio performance than baseline methods, including higher IC/ICIR and Rank IC/ICIR as well as improved AR and Sharpe.

Index Terms—alpha factor mining, programmatic factors, program synthesis, large language models

I. INTRODUCTION

Alpha factor mining is a central objective in quantitative investment, aiming to discover predictive factors that extract actionable signals from noisy, non-stationary market data. Despite decades of research [1]–[4], effective and efficient factor discovery remains challenging due to market complexity, regime shifts, and severe noise. Existing factor mining approaches can generally be divided into two categories: symbolic expression-based methods and neural network-based methods. Symbolic factors [5], [6] built on explicit mathematical expressions provide strong interpretability and clear financial intuition while relying on heavy manual effort and limited scalability. Additionally, symbolic factors tend to be

fragile in the face of rapidly changing market conditions and less adaptive to real-world complexities. Recently, genetic programming (GP) and reinforcement learning (RL) [7]–[9] have enabled automated symbolic factor search within predefined operator spaces, accelerating evolution and preserving a degree of interpretability. However, their strong dependence on manually designed operator sets constrains expressiveness, leading to limited performance and efficiency in practice. Neural network-based [10]–[12] approaches capture implicit patterns and nonlinear relationships in market data. While achieving strong performance, these methods typically suffer from poor interpretability and are prone to overfitting. More recently, large language models (LLMs) have demonstrated remarkable capabilities across a wide range of domains [13], [14], sparking growing interest in their application to alpha mining. AlphaAgent [15] integrates LLM reasoning with financial report knowledge and regularized exploration to mitigate alpha decay, while RD-Agent-Quant (RD-AGENT) [16] proposes an agent-based, data-centric framework for factor and model joint optimization.

However, practical alpha mining requires that signals be not only predictive but also executable, auditable, and efficiently discoverable under practical API and wall-clock budgets. Current methods face three fundamental gaps: **(1) Fixed symbolic grammars limit expressiveness.** Symbolic factors [1], [17] are searched over limited operator spaces, and new operators demand expert-designed semantics and composition rules. The expressiveness boundary is thus permanently human-defined, not search-discovered. **(2) LLM-based methods conflate semantic search with numerical tuning.** Recent LLM-based approaches [15], [16] mix semantic mutation and market-conditioned numerical instantiation into one expensive loop. Effective factors require both computational structure and parameters encoding financial semantics: `corr5` and `corr60` share correlation logic but encode distinct regimes via window lengths. Numerical instantiation demands repeated execution over market data to sense validation fitness landscapes. The

*Corresponding authors

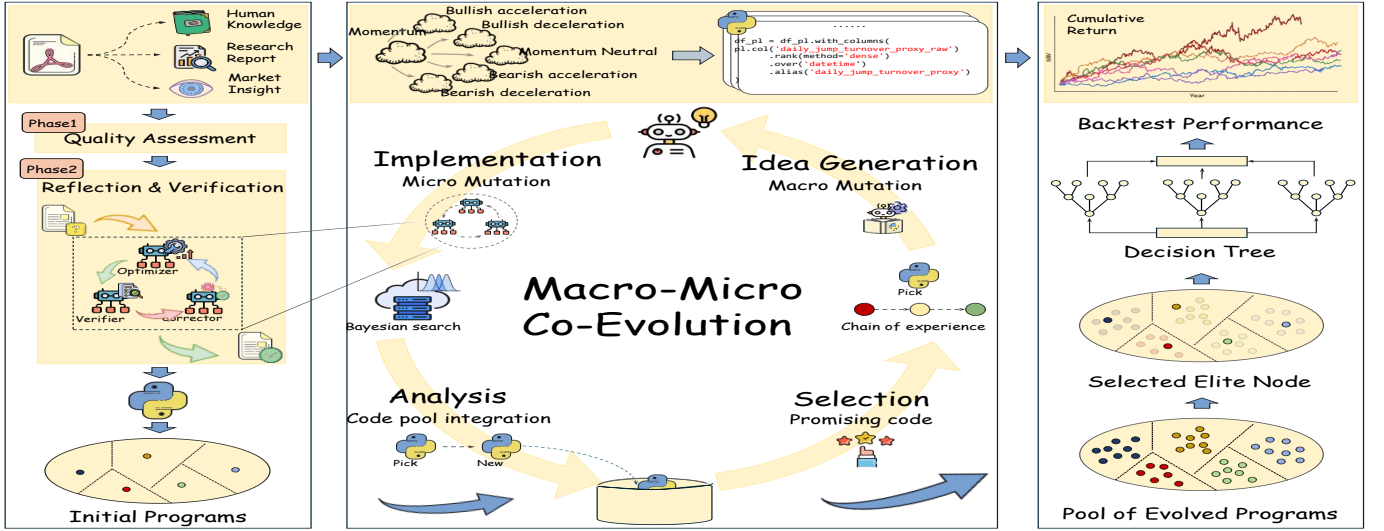


Fig. 1. **Overview of FactorEngine (FE).** *Left: Bootstrapping* extracts factor ideas and converts pseudocode into executable Python to seed a knowledge-infused pool. *Center: Evolution* performs macro–micro co-evolution: LLM agents propose macro mutations guided by chains of experience, and Bayesian search conducts micro-level instantiation with fast validation and feedback updates. *Right: Integration* selects elite factors to train models for backtesting, producing portfolio-level feedback.

conflation wastes API budget on tasks less suited to LLM capabilities and contaminates logic signals with parameter noise. (3) **External domain knowledge remains underutilized.** Financial reports contain multi-step computational logic that compact symbolic grammars do not naturally encode. Existing LLM-based methods that ingest reports often treat them as context stimulation, rather than converting report logic into executable factors.

To address these gaps, we introduce **FactorEngine (FE)**, a programmatic factor discovery framework that represents factors as executable programs satisfying explicit I/O contracts. FE operates at two levels: at the macro level, LLMs propose parameterized logic families (computational structures with variable parameter ranges); at the micro level, Bayesian optimization determines market-conditioned instantiations (which parameter values yield economically validated factors under empirical feedback). This separation shifts expensive and slow LLM reasoning toward cheap and fast local search while preserving semantic exploration. FE further incorporates a bootstrapping module that transforms financial reports into executable seed programs, and a trajectory-aware feedback mechanism that conditions future mutations on execution history.

We evaluate FE on CSI300 and CSI500 using two configurations: **FE-alpha** (initialized with standard handcrafted factors) and **FE-report** (a knowledge-seeded variant initialized from financial reports).

Our contributions are as follows:

- **Program-Level Hyper-Heuristic Framework:** We propose FactorEngine (FE), a system that transforms factor mining into a program evolution problem, extending alpha mining beyond fixed symbolic grammars.
- **Macro-Micro Co-evolution:** We decouple LLM-guided

logic mutation from Bayesian local instantiation, alleviating local optima and efficiency bottlenecks while reducing evolution cost.

- **Knowledge-Infused Factor Diversity:** We propose a closed-loop multi-agent module that operationalizes financial reports into executable seed programs, exploiting prior knowledge grounded in transparent economic rationales.
- **Superior Performance & Diversity:** Extensive experiments demonstrate that FE-alpha surpasses baselines in predictive and portfolio metrics, as well as diversity under matched information budgets. FE-report further evaluates the benefit of report-to-executable bootstrapping, achieving a 58% improvement in Information Coefficient (IC) and a 126% increase in excess annual return compared to Alpha158.

II. RELATED WORK

Traditional Alpha Mining Traditional alpha mining involves handcrafted factors derived from financial domain knowledge, such as Alpha158¹ and Alpha360¹ from Qlib, which are known for their stability and powerful performance. However, manual factor design is labor-intensive and difficult to scale, which motivates automated symbolic approaches. To this end, Genetic programming (GP) methods automatically discover factors with predefined operators. AlphaEvolve [17] further enhances GP with optimization over parameters and matrix-based operations, incorporating AutoML techniques. In parallel, reinforcement learning (RL)-based methods formulate factor mining as a sequential decision-making problem, using financial signals such as Sharpe or Calmar ratios as rewards. AlphaForge [7] adopts a two-stage RL framework to

¹<https://github.com/microsoft/qlib>

discover factor combinations and adaptively adjust weights. Neural factor models [10]–[12], [18] learn latent nonlinear representations from market panels and can capture complex temporal and cross-sectional dependencies. However, their outputs are typically latent model representations rather than explicit, executable factor programs with inspectable logic and parameterized interfaces. FE complements these models by searching and validating program-level factors before their optional use in downstream predictive models.

LLM-based Factor Discovery Recent work applies LLMs to alpha mining. FAMA [19] introduces dynamic factor combination and cross-sample selection to adapt across market regimes. Shi et al. [20] leverages LLM-powered Monte Carlo Tree Search to improve exploration efficiency. AlphaAgent [15] leverages agent-based frameworks to extract inspiration for factor evolution from financial materials, incorporating diversity-aware constraints to mitigate factor decay. RD-Agent [16] proposes a data-centric framework that jointly evolves factors and multi-factor models, achieving an end-to-end automation pipeline that translates model knowledge to symbolic expressions and executable code.

Difference from prior work Although these methods significantly improve performance, they still rely on symbolic representations, restricting expressive power and search space. Furthermore, LLMs in these works are required to handle both logic evolution and parameter optimization, limiting scalability and evolution efficiency. Prior work primarily uses reports as textual inspiration, agent context, or symbolic-expression guidance. In contrast, FE operationalizes report knowledge into executable and testable factor programs under a unified contract. FE also separates LLM-guided logic evolution from local numerical instantiation, reducing LLM/API and wall-clock cost in our evaluated setting.

III. PROBLEM FORMULATION

Consider an N -stock universe observed over T trading days. Let $X_{t-L+1:t} \in \mathbb{R}^{N \times L \times M}$ denote the M raw market-feature channels over a lookback window of length L ending at day t . Let $y_{t+h} \in \mathbb{R}^N$ denotes returns at horizon h .

Mining objective. A factor maps historical features to a cross-sectional score $f(X_{t-L+1:t}) \rightarrow r_t \in \mathbb{R}^N$. Multiple factors $\{f_k\}$ are aggregated by a downstream model g (e.g., LightGBM) for portfolio construction. Factor mining seeks programs and parameters that maximize a predictive metric on validation data:

$$P^*, \theta^* = \arg \max_{P \in \mathcal{P}, \theta \in \Theta(P)} J(P, \theta)$$

where $J(P, \theta)$ can be instantiated as a single-factor validation metric (e.g., IC/RankIC) or as a multi-factor portfolio metric after aggregation by g . FE instantiates $J(P, \theta)$ with the single-factor validation fitness, while portfolio-level metrics are used only for downstream evaluation.

In FE, a programmatic factor is a function $P_\theta : \mathcal{D}_{t-L+1:t} \rightarrow \mathbb{R}^N$ satisfying an execution contract: (i) fixed input schema over structured market panels, (ii) fixed output schema

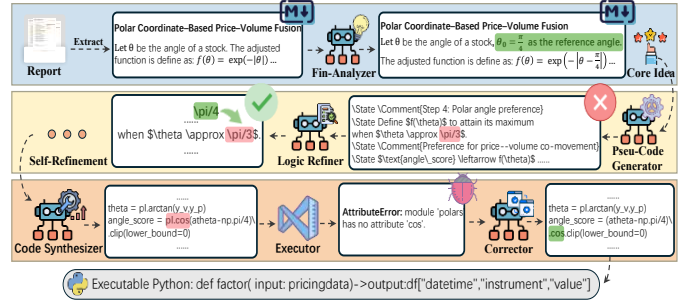


Fig. 2. Overview of the Bootstrapping module with a closed-loop extraction-verification-generation pipeline.

of stock-date-aligned scores, (iii) allowed-library constraints (e.g., pandas, numpy, polars), (iv) explicit parameter schema θ , and (v) leakage-free execution (no look-ahead access). This representation is stronger than fixed symbolic grammars: it permits multi-step preprocessing, conditional logic, and parameterized subroutines while preserving comparability and auditability. We focus on programmatic factors as the fundamental representation for alpha mining.

IV. METHODOLOGY

As illustrated in Fig. 1, the FE system consists of three functionally decoupled yet collaboratively interacting modules, forming a closed-loop pipeline: (1) a Bootstrapping Module, which constructs a knowledge-infused initial factor pool; (2) an Evolution Module, which performs code-level factor evolution guided by chains of experience and empirical feedback signals; and (3) an Integration Module, which supports multi-factor modeling and market data backtesting.

A. Bootstrapping Module

The Bootstrapping Module extracts, refines, and transforms expert knowledge from financial reports and expert-designed factors into programmatic factors. In contrast to traditional symbolic approaches that use reports only as conceptual cues for hypothesis generation, we propose a closed-loop multi-agent system that transforms report-derived knowledge into executable factors, thereby overcoming the limitations of pre-defined expression spaces. As shown in Fig. 2, the Bootstrapping Module consists of three interconnected submodules: (1) PDF Processing: Performs LLM-based compliance screening to retain valid reports, and consolidates the core knowledge with the model’s domain knowledge, yielding reliable inputs for downstream factor extraction. (2) Factor Extraction: Implements a two-step understanding-to-generation workflow with iterative reflection and verification to distill core financial ideas from research reports into the structured JSON representations accompanied by LaTeX-formatted pseudocode. (3) Code Generation: Transforms verified pseudocode and core idea summaries into executable Python code through iterative refinement that validates structural compliance.

Extracted factors, along with their core idea summaries and economic rationales, form a knowledge-infused initial pool

Algorithm 1 Evolution Loop

Require: Knowledge sources $\mathcal{K}$, iterations R , islands I , migration interval M_{mig}

Ensure: Elite factor set $\mathcal{F}^*$

```
1:  $\mathcal{P}_0 \leftarrow \text{BOOTSTRAP}(\mathcal{K})$ 
2: Initialize  $I$  island trees rooted at  $\mathcal{P}_0$ 
3: for  $r = 1$  to  $R$  do
4:   for each island  $i$  in parallel do
5:      $v \leftarrow \text{SELECT}(\text{tree}_i)$ 
6:      $C \leftarrow \text{BUILDCoE}(v)$ 
7:      $(\Delta P, \Theta) \leftarrow \text{LLM-MUTATE}(v, C)$  {Macro}
8:      $P' \leftarrow \text{VALIDATE}(v.\text{code}, \Delta P)$ 
9:      $\theta^* \leftarrow \arg \max_{\theta \in \Theta} \text{fitness}(P', \theta)$  {Micro: instantiation}
10:     $s^* \leftarrow \text{fitness}(P', \theta^*)$ 
11:     $\text{ADDCHILD}(v, (P', \theta^*, s^*)); \text{BACKPROPAGATE}(v, s^*)$ 
12:   end for
13:   if  $r \bmod M_{\text{mig}} = 0$  then
14:      $\text{MIGRATE}(\text{islands}, \text{top-3 per island})$ 
15:   end if
16: end for
17:  $\mathcal{F}^* \leftarrow \{(P, \theta) : \text{fitness}(P, \theta) > \tau\}$ 
18: return  $\mathcal{F}^*$ 
```

for the Evolution Module. This module operationalizes report knowledge as executable and testable programs rather than using reports only as textual inspiration. It also identifies and repairs logical gaps before code execution, producing seed factors that can be evaluated under the same contract as expert-designed alpha factors.

B. Evolution Module

The Evolution Module implements macro-micro co-evolution to address the regime-conditioned nature of alpha factors. At the macro level, LLMs propose parameterized logic families (computational templates with tunable parameter ranges), guided by trajectory-conditioned feedback. At the micro level, local Bayesian instantiation identifies which instantiation of each family is supported by validation data. In alpha mining, parameters such as lookback windows and decay rates encode market horizons and information decay; thus substantially different instantiations can correspond to different economic hypotheses rather than merely performance variants. This separation enables efficient exploration of both computational structure and regime-conditioned instantiations within a unified search framework.

Full evolution loop. As summarized in Algorithm 1, each iteration: (1) *Selection*: choose executable node by UCT [21]; (2) *Mutation context*: build chain-of-experience (CoE) from current path plus diverse high-scoring paths; (3) *LLM proposal*: generate a code patch and parameter search space defining a logic family; (4) *Validation*: check interface compliance, leakage constraints, and executability, repair if needed; (5) *Bayesian instantiation*: select the executable instance (P, θ) supported by validation data using fitness; (6) *Feedback*: store best $(P, \theta, \text{fitness})$, update node statistics, summarize execution results for CoE. Periodically, top nodes are migrated across islands to share discoveries.

To guide evolution efficiently, we define a single-factor fitness metric as a surrogate objective for Bayesian search, node selection, and elite filtering:

$$\text{fitness}(P, \theta) = \frac{1}{4}(10 \cdot IC + ICIR + 10 \cdot RIC + RICIR), \quad (1)$$

which balances linear and rank-based predictive strength and stability on validation data. The sign of fitness records the direction of factor-return association, whereas its magnitude reflects predictive strength. Elite factors selected by fitness are subsequently evaluated using portfolio-level metrics in the Integration Module.

Program Selection. The evolution factor pool is organized as a tree structure, where each node corresponds to an executable program evolved from its parent and can be directly evaluated, yielding an immediate reward equal to its validation fitness. At each iteration, the evolution process selects the most promising node from the current tree as the context for subsequent refinement. We define the node value $Q(v)$ as the empirical mean fitness over all evaluations within the subtree rooted at v including itself, reflecting both its performance and its potential. After each evaluation, $Q(v)$ and the visit count $N(v)$ are updated via backpropagation along the ancestor path. To balance exploration and exploitation, we adopt the Upper Confidence Bound for Trees (UCT) criterion to score candidate nodes. Specifically, the value of a node is:

$$\text{UCT}(v) = Q(v) + c \sqrt{\frac{\ln N_{\text{parent}(v)}}{N_v}}, \quad (2)$$

where $N_{\text{parent}(v)}$ denotes the visit count of v 's parent, and c is a constant that controls the exploration–exploitation trade-off. We set $c = \sqrt{2}$ as it is commonly used. Unlike standard MCTS settings where nodes represent partial states or intermediate decisions, each node in our tree is a fully specified program and thus independently executable and evaluable, enabling flexible evolution.

Idea Generation. We prompt the agent to synthesize environmental feedback and its parametric knowledge to generate high-level inspirations and structural modifications of programs. This process emphasizes semantic reasoning, pattern abstraction, and conceptual exploration, where the LLM excels, realizing **macro mutations** as shown in Fig. 1.

Based on the selected program node, we construct the current chain of experience (CoE) C as the sequence of program nodes on the historical trajectory leading to that node. From the global tree, we further select $n = 3$ candidate paths $\{p_i\}_{i=1}^n$ that jointly balance high empirical performance and low overlap with C . Each path serves as a compact representation of prior evolution experience. Formally, given C , let $p_i = \{p_{i,1}, p_{i,2}, \dots, p_{i,|p_i|}\}$ denote a candidate path, and let $\Phi_i = p_i \cap C$ be the set of shared program nodes between p_i and C . We define the overlap score as:

$$S_{\text{cov}}(p_i) = \frac{|\Phi_i|}{|p_i|} + \frac{|\Phi_i|}{|C|} \quad (3)$$

which symmetrically penalizes both candidate-path overlap and current-chain overlap. Let $P_{i,m}$ denote the program stored

at node $p_{i,m}$, and let $\theta_{i,m}^* = \arg \max_{\theta \in \Theta(P_{i,m})} \text{fitness}(P_{i,m}, \theta)$ be its best validation-supported instantiation. The effectiveness score of a path is defined as the average fitness of all best-instantiated nodes on the path:

$$S_{eff}(p_i) = \frac{1}{|p_i|} \sum_{m=1}^{|p_i|} \text{fitness}(P_{i,m}, \theta_{i,m}^*). \quad (4)$$

The final path score of p_i is then computed as:

$$S_{total}(p_i) = S_{eff}(p_i) - S_{cov}(p_i) \quad (5)$$

These selected paths are then assembled into a structured context, which exposes explicit experience knowledge to the LLM and cooperates with its intrinsic parametric knowledge to generate a mutation idea. In practice, evolutionary optimization is inherently non-monotonic. Consequently, FE explicitly captures the full dynamic process, including transient fluctuations and local setbacks, rather than just the final success. Unlike methods that condition only on static high-performing nodes, CoE conditions future mutation proposals on both successful and unsuccessful trajectories, helping the agent avoid previously degraded directions and reuse empirically useful structural changes.

Implementation. In this stage, FE performs **micro mutations**: data-driven numerical instantiation of a proposed factor logic family. Local Bayesian instantiation employs Bayesian search, with our implementation supporting the Tree-structured Parzen Estimator (TPE). For a given evolved program P with parameter vector $\theta \in \Theta$, FE solves $\theta^* = \arg \max_{\theta \in \Theta} \text{fitness}(P, \theta)$. These Bayesian methods model the objective function probabilistically and suggest parameters that maximize Expected Improvement: $EI(\theta) = \int_{-\infty}^{\infty} \max(y^* - y, 0) \cdot p(y|\theta) dy$, where y^* is a performance threshold (typically the top 25% of observed scores), balancing exploration of uncertain regions and exploitation of promising regions to efficiently converge to optimal parameter combinations. During the Idea Generation phase, the LLM agent specifies parameter search ranges (e.g., window sizes, decay factors) based on domain knowledge and previous results, but the actual parameter exploration is delegated to local Bayesian search.

The validation process employs a two-phase strategy to operationalize the resource separation: - **Phase 1**: Sequential code validation with LLM-based automatic correction, ensuring evolved programs execute correctly before parallel optimization. - **Phase 2**: Bayesian micro-instantiation entirely on local computational resources without LLM calls. This phase employs multi-process parallel execution, distributing trials across multiple workers. The Bayesian search algorithm coordinates candidate parameters to reduce redundant exploration. The evaluation function $\text{fitness}(P, \theta)$ computes empirical performance metrics through high-throughput execution on historical market data. This phase uses cached market data, multi-process execution, and efficient IC/RIC aggregation across lag periods (1, 3, 5, 10 days). This design reports local evaluations transparently while treating LLM/API calls and wall-clock time as the primary scarce resources.

Feedback propagation. After a new program is instantiated and evaluated, the framework performs feedback propagation to transform empirical outcomes into actionable guidance for subsequent evolution. Specifically, the LLM is prompted to summarize the core implementation logic of the new program, identify its relative changes from the root and the parent program, as well as assess its performance improvement or degradation. These elements are jointly organized into a structured format and stored within the node for experience reuse in later iterations. Concurrently, the quantitative feedback is propagated back along the evolution path, updating the Q and N values of all traversed nodes up to the root.

Through the integration of these four stages, FE supports trajectory-conditioned mutation, macro–micro coordinated evolution, and executable validation.

Despite forming a complete evolution loop, LLM-driven mutation proposals remain stochastic. Consequently, even under identical configurations, individual evolution trajectories may diverge substantially in both structure and performance, potentially leading to inefficient exploration. To address this issue, we introduce a multi-island evolution configuration upon the FE framework to improve both efficiency and sampling diversity. Specifically, we initialize I independent evolution processes by duplicating the initial program node and evolve them concurrently, each of which acts as an isolated island. Every M_{mig} evolution rounds, each island selects its top-3 programs by fitness and migrates the copies to other islands as child nodes of the target island’s root node. Migrated programs enter later context construction, allowing useful structural changes discovered in one trajectory to be reused by others while preserving diversity.

C. Integration Module

Building upon the bootstrapping and evolution modules, the integration module is designed to construct high-performing multi-factor signals from the evolving factor pool via elite node selection and multi-factor modeling, targeting robust predictive performance in real-world financial markets.

While explicitly modeling factor correlations with the existing factor pool is useful, the computational cost of dependency analysis grows rapidly as the factor pool expands. We therefore use a threshold-based filter: for each initial seed, we retain at most the top 5 evolved nodes whose fitness exceeds 0.4 under a rolling evaluation window of length $L = 2$. For each retained node, we select the top 10 parameter configurations by the same criterion. These generated factors are then merged with Alpha158 to train the downstream LightGBM model; the final portfolio metrics evaluate complementary alpha rather than standalone generated-factor performance.

V. EXPERIMENT SETUP

Baselines. We compared FactorEngine (FE) with several representative baseline methods: (1) GPLEarn [1], which performs symbolic factor discovery via genetic programming; (2) traditional time-series forecasting neural models, such as LightGBM (LGBM), LSTM and Transformer, which capture

temporal dependencies in the market data; (3) specialized financial model, TRA [22], which focuses on integrating multiple trading strategies and modeling non-i.i.d. market patterns; (4) agent-based alpha factor mining methods, including AlphaAgent [15] and RD-Agent-Quant [16] (RD-Agent); (5) the hand-crafted factor baseline Alpha158. For FE, we ran two variants. **FE-alpha** starts from expert-designed alpha factors and is the primary information-budget-matched comparison with AlphaAgent and RD-Agent. **FE-report** starts from financial reports and is a knowledge-seeded variant for evaluating report-to-executable bootstrapping. The report seeds are derived from Chinese sell-side quantitative research reports. To prevent potential leakage in the knowledge-infused bootstrapping module, we only used financial research reports published before 2017 for factor extraction and code bootstrapping, ensuring that no report content overlaps with the test period. We acknowledge that modern LLMs are trained on data extending beyond our test period. However, this limitation is shared by all agent-based methods, and we ensured fair comparison by using Gemini-2.5-Pro as the backbone model across all agent baselines, with the temperature set to 0.7.

Hyperparameters. We conducted two experiments with different budgets, in which each framework evolved 200 and 400 iterations respectively, with one factor generated per iteration. Under the two budgets, the FE framework was initialized with 5 and 10 alpha factors (or reports), respectively. For FE-alpha, the 5-factor seed set is {corr5, resi5, klen, klow, vstd5}, and the 10-factor seed set is {corr5, resi10, roc60, rsqr5, cord5, std5, klen, klow, vstd5, wvma5}. For FE-report, each seed factor is derived from a single report through a conversion process. The number of islands was set to 2, with migration performed every 7 iterations. For each FE macro mutation, local Bayesian instantiation runs 20 trials over the LLM-proposed parameter ranges without additional LLM calls. All generated factors were filtered according to each framework’s criteria before multi-factor modeling. For backtesting, generated factors from all mining methods were merged with the Alpha158 set to train a LightGBM model. This setting follows common multi-factor practice and evaluates whether generated factors provide

complementary alpha over a strong standard factor base.

Trading protocol. Portfolio metrics are net of realistic A-share costs. The strategy ranks stocks by model score, buys the top 50, and uses five rolling 5-day sub-portfolios with equal weights, following the same portfolio construction and ranking scheme as all baselines. We charge commission 1.5×10^{-4} on both buys and sells, stamp duty 5×10^{-4} on sells, and slippage 8×10^{-4} on all trades. Execution also enforces a 1-lot minimum, a 10% daily-volume cap per stock, price-limit constraints, and initial capital of 100M CNY. For multi-factor modeling, the LightGBM model is retrained under an “8+1+1” walk-forward protocol.

Datasets. We used the full-market dataset, divided into training (2008-01-01 – 2014-12-31), validation (2015-01-01 – 2016-12-31), and testing (2017-01-01 – 2024-12-31) periods for factor discovery, and evaluated in the CSI300 and CSI500 markets. The raw features used for factor construction consist solely of OHLCV data.

Metrics. For predictive performance, we reported the Information Coefficient (IC), Information Coefficient Information Ratio (ICIR), Rank IC, and Rank ICIR. For portfolio performance, we evaluated Annualized Return (AR), Information Ratio (IR), maximum drawdown (MDD) and Sharpe Ratio (SR). Portfolio metrics are computed on the excess return series, defined as the portfolio return minus the benchmark return.

All experiments were conducted on a server equipped with 56 CPU cores, providing a total of 56 parallel threads.

VI. RESULTS ANALYSIS

A. Main Result

Tab. I presents the experimental results of FactorEngine (FE) and baseline methods in the CSI300 and CSI500 markets.

Traditional neural models like LGBM, LSTM, and Transformer show weaker predictive and portfolio results in this setting, especially on CSI500 where their excess returns are negative. The TRA method used artificial factors for deep learning modeling, integrating their feature representations, which significantly enhanced the predictive capability of the RNN network. The 50 factors generated by GPLEarn show

TABLE I

BOLD AND UNDERLINING DENOTE THE BEST AND SECOND-BEST RESULTS, RESPECTIVELY; "-1"/"-2" DENOTE 200/400 ITERATIONS. FE-ALPHA IS THE FAIR COMPARISON, WHILE FE-REPORT USES REPORT-DERIVED SEEDS.

Methods	CSI300								CSI500							
	IC	ICIR	RIC	RICIR	AR	MDD	IR	SR	IC	ICIR	RIC	RICIR	AR	MDD	IR	SR
LGBM	0.0040	0.0326	0.0078	0.0587	0.0129	39.18%	0.1706	0.1006	0.0057	0.0531	0.0108	0.0913	-0.0514	48.22%	-0.3317	-0.3051
LSTM	0.0053	0.0313	0.0129	0.0704	0.0486	34.09%	0.4810	0.3241	0.0054	0.0306	0.0138	0.0726	-0.0104	36.05%	-0.0080	-0.1156
Transformer	-0.0012	-0.0066	-0.0078	-0.0388	0.0342	29.48%	0.3027	0.1490	0.0012	0.0076	-0.0024	-0.0130	-0.0221	53.04%	-0.0542	-0.1598
TRA	0.0256	0.1559	0.0302	0.1964	0.0674	16.02%	0.6881	0.3747	0.0341	0.3184	0.0295	0.2717	0.0320	31.62%	0.2877	0.0072
Alpha158	0.0299	0.2008	0.0331	0.2164	0.0840	17.49%	0.7440	0.4196	0.0403	0.3100	0.0416	0.3172	0.0197	25.17%	0.2152	0.0089
GPLEarn	0.0292	0.1971	0.0321	0.2120	0.0814	<u>15.99%</u>	0.7337	0.4152	0.0409	<u>0.3190</u>	0.0427	0.3113	0.0272	<u>22.79%</u>	0.2751	0.0451
RD-Agent-1	0.0255	0.1667	0.0294	0.1881	0.0507	23.71%	0.4770	0.2627	0.0385	0.2887	0.0398	0.2963	-0.0033	30.55%	0.0404	-0.0915
AlphaAgent-1	0.0282	0.1978	0.0313	0.2142	0.0673	17.00%	0.6346	0.3499	0.0400	0.3076	0.0407	0.3135	0.0102	24.40%	0.1431	-0.0319
FE-alpha-1	0.0319	0.2178	0.0346	0.2308	0.0888	16.91%	0.7886	0.4526	0.0413	0.3112	0.0430	0.3241	0.0346	21.44%	0.3315	0.0793
FE-report-1	0.0333	0.2325	0.0360	0.2459	0.1017	15.89%	0.8959	0.5147	0.0420	0.3244	<u>0.0429</u>	0.3289	0.0458	25.44%	0.4030	0.1222
RD-Agent-2	0.0269	0.1833	0.0300	0.1978	0.0917	15.23%	0.8113	0.4647	0.0402	0.3070	0.0411	0.3130	0.0189	24.15%	0.2092	0.0052
AlphaAgent-2	0.0314	0.2089	<u>0.0346</u>	0.2252	0.0755	16.23%	0.6779	0.3868	0.0385	0.2870	0.0396	0.2906	0.0235	25.36%	0.2437	0.0266
FE-alpha-2	<u>0.0315</u>	<u>0.2211</u>	0.0344	<u>0.2360</u>	<u>0.0943</u>	<u>15.07%</u>	<u>0.8241</u>	0.4762	<u>0.0417</u>	<u>0.3183</u>	0.0434	<u>0.3293</u>	<u>0.0399</u>	<u>23.84%</u>	<u>0.3770</u>	0.1064
FE-report-2	0.0474	0.3185	0.0475	0.3146	0.1899	12.61%	1.6001	1.0093	0.0536	0.4140	0.0487	0.3744	0.0836	21.51%	0.6719	0.2945

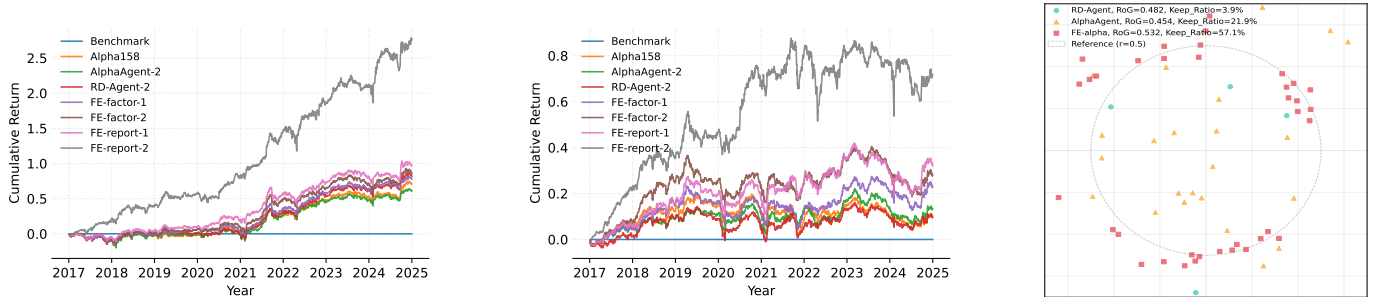


Fig. 3. (Left/Middle) Net cumulative excess returns in CSI300 and CSI500 under the cost-aware trading protocol. (Right) Factor-pool diversity diagnostic on CSI500 via MDS on $1 - |\rho|$, with RoG and keep ratio reported in the legend.

that, in the CSI300 market, mixing these 50 factors with Alpha158 does not further improve the correlation between the features and future returns. However, in the CSI500 market, GPLEARN’s factors provide significant returns. FE-alpha provides the primary fair comparison with agent-based baselines and consistently improves predictive and portfolio metrics under the same initialization budget. FE-report further improves performance when pre-2017 report-derived seeds are available, evaluating the end-to-end utility of report-derived initialization. In our experiments, although three agent-based methods showed improvements as the number of iterations increased, AlphaAgent and RD-Agent performed noticeably worse than the Alpha158 factor in the CSI300 market with fewer iterations. However, as the number of iterations increased, the performance gap narrowed, and they eventually surpassed Alpha158. In contrast, both FE setups consistently showed better predictive performance and portfolio performance than other methods at different iteration rounds. This indicates that the program-level factor evolution framework (FE) is effective at discovering factors that better capture market characteristics. Furthermore, when the number of report-derived seed factors increased from 5 to 10, the performance of the factors evolved from financial reports significantly improved. IC increased from 0.0333 to 0.0474, AR improved from 0.1017 to 0.1899, and MDD decreased from 15.89% to 12.61%. It shows that FE effectively extracts financial knowledge from reports for modeling and continuously evolves factors during iterations.

Through the two FE experimental setups, we found that the factors evolved from reports consistently outperformed those

evolved from expert-designed alpha seeds, regardless of the iteration rounds. The results suggest that report-derived prior knowledge can provide useful initialization for factor evolution, although the separate contribution of the bootstrapping stages is not isolated in this comparison. Fig. 3 shows the net cumulative excess-return curves under the common cost-aware trading protocol in the CSI300 and CSI500 markets. The factors evolved using FE outperformed Alpha158, AlphaAgent, and RD-AGENT in both 200-iteration and 400-iteration experiments.

B. Factor Diversity Analysis

To evaluate the non-redundancy of generated factors, we first filter out low-quality factors with an IC below 0.015 and then analyze the correlation structure of the retained factor pool. We construct a dissimilarity matrix with entries $1 - |\rho|$ and use Multidimensional Scaling (MDS) to embed it into a 2D space. The absolute correlation treats two factors with opposite signs but near-identical rankings as redundant, so the visualization focuses on signal novelty rather than sign direction. In the embedded space, larger distances indicate weaker absolute correlations between factors. As shown in the right subfigure of Fig. 3, in the CSI500 setting, FE-alpha retained 36 effective factors after quality filtering, corresponding to a 57.1% keep ratio and significantly surpassing the yield of AlphaAgent and RD-AGENT. The legend further reports the Radius of Gyration (RoG), which summarizes the overall dispersion of the retained factors in the MDS space. Beyond the number of surviving factors, FE-alpha exhibits broader dispersion in the MDS diagnostic and the largest RoG,

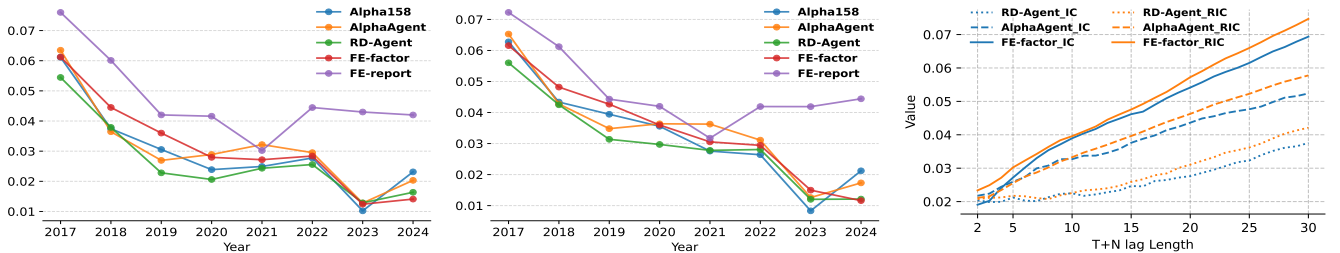


Fig. 4. Yearly IC (Left) and Rank IC (Middle) comparisons on the CSI300 market. Mean IC and Rank IC between the top 10% factors and future returns at $T+N$ on the CSI300 market across three settings (Right; the plot label “ $T+N$ ” denotes an N -day-ahead return horizon).

indicating that its retained factors are not concentrated in a narrow correlation cluster. Together, the higher keep ratio and larger dispersion suggest that FE-alpha produces a richer and less redundant factor set, offering more complementary alpha signals for downstream multi-factor models.

C. Alpha Decay Analysis

In Fig. 4, the first two subplots show the yearly IC variation trends on the test data of the CSI300 market under the 400-iteration setting, with Alpha158 included as the handcrafted baseline. All factors exhibited some degree of decay over time, but throughout the entire period, FE-report consistently maintained a high IC and Rank IC. After 2021, FE-report no longer followed the earlier decay trend and showed a recovery in yearly IC/Rank IC, suggesting better adaptation to the later test-period market conditions. The FE-alpha experimental group experienced a more gradual decay compared to other factors, with no significant fluctuations, resulting in superior cumulative returns and portfolio performance. Although the IC values of other methods were slightly higher in some years, their significant declines in other years make it difficult to mitigate the losses. The right panel of Fig. 4 provides a single-factor performance without LightGBM aggregation. We select the top 10% generated factors from each agent-based method after 400 evolution rounds and compute their IC and Rank IC against forward returns at different horizons. FE consistently achieves higher IC/Rank IC across horizons, indicating that its discovered factor pool contains stronger predictive signals rather than only improving through downstream LightGBM aggregation. The stable advantage across lag lengths further suggests that FE factors are not over-specialized to a single return horizon, but capture more robust cross-sectional signals.

D. Robustness of Factor Evolution

Because LLM-guided evolution is non-deterministic, final portfolio metrics alone do not reveal whether factor discovery is stable across starting conditions. We therefore analyze FE-alpha trajectories across 20 initial seed programs on CSI300 from two complementary views. Fig. 5 illustrates how $|\text{fitness}|$ varies with iteration and generation. Here, generation denotes tree depth in the evolution tree: the root seed is Gen-0, and each mutation edge increases the generation by one. For a separate stability diagnostic, we evolved 20 alpha seeds to estimate across-seed dispersion more reliably. Unlike the mining-stage fitness used for node selection, the diagnostics in Fig. 5 report $|\text{fitness}|$ on the held-out test data. The right plot pools observed generation depths from the multi-seed FE-alpha diagnostic; Gen-8 is the maximum observed depth in the aggregated evolution results, not a depth reached by every seed trajectory. The Case Study below is a separate single Abandoned-Baby checkpoint-40 tree whose deepest observed generation is 4. The trajectories are non-monotonic, as expected for mutation-based search, but the descriptive statistics show an upward shift in factor quality while preserving bounded dispersion. In the right plot, the mean $|\text{fitness}|$ increases from about 0.08 at Gen-0 to about 0.13 at Gen-8,

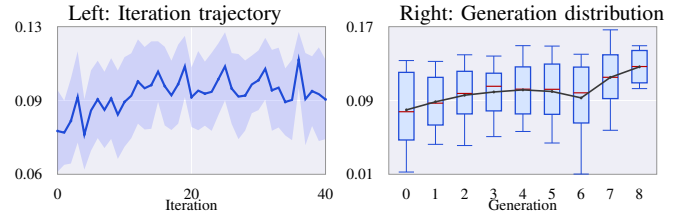


Fig. 5. Evolution stability of FE-alpha on CSI300. Left: mean test-set $|\text{fitness}|$ trajectory across 20 initial seed programs, with shading indicating across-seed dispersion. Right: generation-level test-set $|\text{fitness}|$ distribution across observed nodes.

TABLE II
SYSTEM RUNTIME AND EXECUTABILITY UNDER THE 200-ITERATION SETTING. "DEBUG" IS THE RATIO OF API CALLS USED FOR CODE DEBUGGING.

Methods	Search Space	Run Space	Cost(\$)	Time(h)	Executable Ratio	Debug Ratio
RD-Agent	symbolic	code	16.91	48.0	96%	68%
AlphaAgent	symbolic	code	11.61	9.7	93%	51%
FactorEngine	code	code	12.01	0.5	99%	32%

while the inter-quartile range narrows from about 0.07 to 0.04. This provides factor-discovery-level robustness evidence that complements the downstream portfolio results. This analysis targets factor-discovery stability across starting factor families rather than repeated full-portfolio retraining.

E. Token Efficiency and Executability

Tab. II reports system-level efficiency under our implementation for three agent-based frameworks running 200 iterations. Baselines are evaluated using their official public implementations under identical hardware. The primary scarce resources are LLM/API calls and wall-clock time. FE reduces pressure on these resources by separating LLM-guided logic mutation from local numerical instantiation: the 20 Bayesian trials per macro mutation are executed without extra LLM calls and are included in the reported wall-clock time. The efficiency gains stem from two complementary sources: (1) the architectural separation that eliminates redundant LLM calls for parameter search and expression-to-code conversion, and (2) system-level implementation choices that minimize local evaluation latency. Both are integral to the FE framework. FE also achieves the highest executable ratio (99%), reflecting the benefit of evolving directly within a validated code contract.

F. Ablation Study

Bayesian Micro-Instantiation vs. Fixed Instantiation. We ablated the micro-level instantiation mechanism under the same macro-level evolution budget (40 iterations): **w/ Bayes** applied local Bayesian instantiation to each LLM-proposed factor logic family before evaluation, while **w/o Bayes** used fixed/default parameters. The left panel of Fig. 6 shows that Bayesian search improves both *final quality* and *search speed*. Concretely, the best evolved program after 40 iterations was substantially higher with Bayesian instantiation (about 0.38 vs. 0.25). Moreover, the improvement trajectory was consistently steeper, suggesting that validation-supported numerical

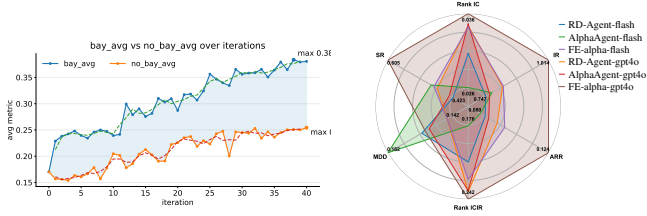


Fig. 6. **Left:** Effect of Bayesian micro-instantiation. `bay_avg` uses local Bayesian instantiation, while `no_bay_avg` uses fixed/default parameters. **Right:** Backbone robustness across GPT-4o and Gemini-2.5-flash-lite.

TABLE III
COMPARISONS OF FE-ALPHA UNDER DIFFERENT PROMPT SETTINGS, ISLAND CONFIGURATIONS, AND NUMBERS OF INITIAL FACTORS.

config	RIC	RICIR	AR	IR	MDD
5alpha,1island,CoE	0.0325	0.2165	0.0728	0.6737	0.1678
5alpha,1island,top-k	0.0319	0.2125	0.0696	0.6428	0.1626
5alpha,2island,CoE	0.0346	0.2308	0.0888	0.7886	0.1691
5alpha,2island,top-k	0.0332	0.2189	0.0775	0.7085	0.1614
10alpha,1island,CoE	0.0344	0.2358	0.0782	0.7079	0.1787
10alpha,1island,top-k	0.0341	0.2266	0.0761	0.6944	0.1673
10alpha,2island,CoE	0.0344	0.2360	0.0943	0.8241	0.1507
10alpha,2island,top-k	0.0353	0.2328	0.0839	0.7648	0.1708

instantiation provides a stronger and less noisy fitness signal for promoting promising program logic.

Backbone Ablation. We conducted experiments using three agent-based frameworks with the Gemini-2.5-Flash-Lite² and GPT-4o³ for backbone ablation on the CSI300 market data. As shown in the right panel of Fig. 6, all frameworks evolved for 200 iterations. Since GPT-4o generally exhibits stronger reasoning capabilities, the alpha factors generated by GPT-4o exhibited stronger predictive and portfolio performance. Among the six setups, FE-alpha with GPT-4o showed the best results in Rank IC, Rank ICIR, AR, IR, and SR.

Configuration Ablation. We further evaluate CoE prompts and multi-island evolution as search configurations that complement the core macro-micro mechanism. Tab. III evaluates their effect under different numbers of initial factors. Across both 5-alpha and 10-alpha settings, two-island configurations generally improve portfolio metrics relative to one-island configurations, suggesting that island diversification helps stabilize exploration. CoE prompts also improve several predictive and portfolio metrics, indicating that trajectory-conditioned context provides useful guidance.

G. Case Study

We provide an example from the *Abandoned Baby* report seed to illustrate how FE evolves a continuous, executable, and auditable factor from a report over 40 evolution iterations. The root program and the representative evolved node below are both taken from the same evolution tree. The root node has a fitness of 0.1006, while the selected evolved node, found at iteration 40 and generation depth 4, reaches 0.3935. Due

to space, the listings show only factor-logic-related content and omit non-essential pre- and post-processing steps. Both programs still follow the same executable output contract.

Core Root Program Excerpt

Listing 1. Root Abandoned-Baby program excerpt.

```
def trend_factor(pricing_data: pl.DataFrame, parameters):
    # Omitted: imports, I/O preprocess, Parameter Definition
    # Root: direct three-candle continuous pattern scoring.
    df_pl = df_pl.with_columns([
        pl.col(c).shift(1).over('instrument').alias(f'{c}_lag1')
    ])
    for c in ['open', 'close', 'high', 'low']:
        pl.col(c).shift(2).over('instrument').alias(f'{c}_lag2')
    for c in ['open', 'close', 'high', 'low']:
        r2 = pl.col('high_lag2') - pl.col('low_lag2') + epsilon
        bull = (pl.col('close_lag2') - pl.col('open_lag2')) / r2
        r1 = pl.col('high_lag1') - pl.col('low_lag1') + epsilon
        body = (pl.col('open_lag1') - pl.col('close_lag1')).abs() / r1
        doji = (1.0 - body / body_threshold).clip(0.0, 1.0)
        gup = ((pl.col('low_lag1') - pl.col('high_lag2')) / (pl.col('high_lag1') + epsilon)).clip(-0.5, 0.5) + 0.5
        rt = pl.col('high') - pl.col('low') + epsilon
        bear = (pl.col('open') - pl.col('close')) / rt
        gdn = ((pl.col('low_lag1') - pl.col('high')) / (pl.col('low_lag1') + epsilon)).clip(-0.5, 0.5) + 0.5
        # Rolling context keeps this as executable logic.
        vavg = pl.col('volume').rolling_mean(window_size=vol_lookback, min_periods=1).over('instrument')
        vol = ((pl.col('volume') / (vavg + epsilon)) / vol_multiplier).clip(0.0, 2.0)
        pq = pl.col('close').rolling_quantile(quantile=price_quantile, window_size=price_lookback, min_periods=1).over('instrument')
        px = (pl.col('close') / (pq + epsilon)).clip(0.5, 1.5)
        F = -(0.15 * bull + 0.20 * doji + 0.15 * gup + 0.20 * bear + 0.15 * gdn + 0.075 * vol + 0.075 * px)
        df_factor = df_pl.with_columns(F.cast(pl.Float64)).alias('TrendFactor')
    # Omitted: select columns, filter invalid values.
    return df_factor
```

Core Evolved Program Excerpt (Iteration 40)

Listing 2. Evolved Abandoned-Baby program excerpt.

```
def trend_factor(pricing_data: pl.DataFrame, parameters):
    # Omitted: imports, I/O preprocess, Parameter Definition
    # Evolved: adds ROC context, ATR scaling, exponential doji, range position.
    df_pl = df_pl.with_columns([
        pl.col('close').shift(1).over('instrument').alias('close_lag1')
    ])
    tr = pl.max_horizontal(pl.col('high') - pl.col('low'), (pl.col('high') - pl.col('close_lag1')).abs(), (pl.col('low') - pl.col('close_lag1')).abs())
    df_pl = df_pl.with_columns(tr.ewm_mean(span=atr_lookback, min_periods=atr_lookback).over('instrument').alias('atr'))
    df_pl = df_pl.with_columns([pl.col(c).shift(1).over('instrument').alias(f'{c}_lag1')
    ])
    for c in ['open', 'close', 'high', 'low', 'atr']:
        pl.col(c).shift(2).over('instrument').alias(f'{c}_lag2')
    for c in ['open', 'close', 'high', 'low', 'atr']:
        p0 = pl.col('close_lag2').shift(roc_lookback).over('instrument')
        mom = ((pl.col('close_lag2') - p0) / (p0 + epsilon)).clip(0, 1.0)
        r1 = pl.col('high_lag1') - pl.col('low_lag1') + epsilon
        body = (pl.col('open_lag1') - pl.col('close_lag1')).abs() / r1
        doji = (-doji_exp_decay * body).exp()
        gup = ((pl.col('low_lag1') - pl.col('high_lag2')) / (pl.col('atr_lag2') + epsilon)).clip(0, 1.5)
        bear = ((pl.col('open') - pl.col('close')) / (pl.col('atr') + epsilon)).clip(0, None)
        gdn = ((pl.col('low_lag1') - pl.col('high')) / (pl.col('atr_lag1') + epsilon)).clip(0, 1.5)
```

²<https://ai.google.dev/gemini-api/docs/models>

³<https://openai.com/index/gpt-4o-system-card/>

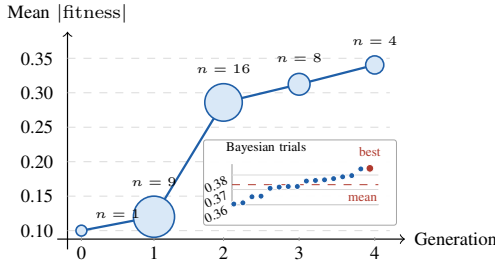


Fig. 7. **Abandoned-Baby evolution quality.** Bubbles show mean training |fitness| by generation, with radius encoding within-generation standard deviation. The inset reports valid Bayesian-instantiation scores for the selected generation-4 node.

```
vavg = pl.col('volume').rolling_mean(window_size=
    vol_lookback,
    min_periods=1).over('instrument')
vol = ((pl.col('volume') / (vavg + epsilon)) /
    vol_multiplier).clip(0.0, 2.0)
lo = pl.col('low').rolling_min(window_size=
    price_lookback,
    min_periods=price_lookback // 2).over('instrument')
hi = pl.col('high').rolling_max(window_size=
    price_lookback,
    min_periods=price_lookback // 2).over('instrument')
px = ((pl.col('close') - lo) / (hi - lo + epsilon)).clip(
    0.0, 1.0)
F = -(0.25 * mom + 0.20 * doji + 0.15 * gup +
    0.10 * bear + 0.15 * gdn + 0.075 * vol + 0.075 * px)
df_factor = df_pl.with_columns(F.cast(pl.Float64).alias
    ('TrendFactor'))
# Omitted: select columns, filter invalid values.
return df_factor
```

This example presents a report-inspired alpha whose evolved program acts as a structured candlestick-pattern interpreter, combining lagged candle semantics, true range and EWM-based ATR volatility normalization, exponential doji-quality scoring, ATR-scaled gap strength, ROC-based prior momentum, and volume/price-position conditioning. The factor remains continuous and executable under the seed program’s output contract. Fig. 7 shows Bayesian micro-instantiation trials for the selected generation-4 evolved program in Listing 2: 15 out of 20 trials produced valid scores, while invalid trials were discarded due to execution and contract checks.

VII. CONCLUSION

We presented *FactorEngine*, a program-level alpha factor mining framework for discovering *executable and auditable* factors while keeping the evolution tractable. FE departs from prior symbolic expression search by representing factors as execution-constrained programs and separating logic evolution from market-conditioned instantiation. FE further introduces a knowledge-infused module that transforms financial reports into executable seeds via a closed-loop extraction–verification–generation pipeline. CoE prompts and island search act as supporting mechanisms for trajectory-conditioned context and diversity-preserving exploration. Overall, FE provides an empirical framework for factor discovery in discrete program spaces, where LLM-guided structural mutation is paired with local Bayesian micro-instantiation under validation feedback. Future work includes extending to richer data modalities, improving robustness

under distribution shift and enabling the LLM to actively interrogate market data.

REFERENCES

- [1] T. Stephens, “gplearn: Genetic programming in python,” <https://github.com/trevorstephens/gplearn>, 2016.
- [2] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [3] X. Zhang, P. Li, J. Zhu, and J. Tang, “Temporal routing adaptor for deep time series forecasting,” in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022, pp. 2447–2457.
- [4] A. Vaswani, N. Shazeer, N. Parmar *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [5] E. F. Fama and K. R. French, “The cross-section of expected stock returns,” *the Journal of Finance*, vol. 47, no. 2, pp. 427–465, 1992.
- [6] K. Hou, C. Xue, and L. Zhang, “Replicating anomalies,” *The Review of Financial Studies*, vol. 33, no. 5, pp. 2019–2133, 2020.
- [7] H. Shi, W. Song, X. Zhang, J. Shi, C. Luo, X. Ao, H. Arian, and L. A. Seco, “Alphaforge: A framework to mine and dynamically combine formulaic alpha factors,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 12, 2025, pp. 12 524–12 532.
- [8] S. Yu, H. Xue, X. Ao *et al.*, “Generating synergistic formulaic alpha collections via reinforcement learning,” in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023, pp. 5476–5486.
- [9] T. Zhang, Y. Li, Y. Jin, and J. Li, “Autoalpha: An efficient hierarchical evolutionary algorithm for mining alpha factors in quantitative investment,” *arXiv preprint arXiv:2002.08245*, 2020.
- [10] Y. Duan, L. Wang, Q. Zhang, and J. Li, “Factorvae: A probabilistic dynamic factor model based on variational autoencoder for predicting cross-sectional stock returns,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 36, no. 4, 2022, pp. 4468–4476.
- [11] W. Xu, W. Liu, C. Xu, J. Bian, J. Yin, and T.-Y. Liu, “Rest: Relational event-driven stock trend forecasting,” in *Proceedings of the web conference 2021*, 2021, pp. 1–10.
- [12] W. Xu, W. Liu, L. Wang, Y. Xia, J. Bian, J. Yin, and T.-Y. Liu, “Hist: A graph-based framework for stock trend forecasting via mining concept-oriented shared information,” *arXiv preprint arXiv:2110.13716*, 2021.
- [13] T. B. Brown *et al.*, “Language models are few-shot learners,” *NeurIPS*, 2020.
- [14] J. Wei *et al.*, “Emergent abilities of large language models,” *arXiv preprint arXiv:2206.07682*, 2022.
- [15] Z. Tang, Z. Chen, J. Yang *et al.*, “Alphaagent: Llm-driven alpha mining with regularized exploration to counteract alpha decay,” in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Min. (KDD)*, 2025, pp. 2813–2822.
- [16] Y. Li, Y. Xu, Y. Xiao, M. Xu, X. Wang, W. Liu, and J. Bian, “R&d-agent-quant: A multi-agent framework for data-centric factors and model joint optimization,” *arXiv preprint arXiv:2505.15155*, 2025.
- [17] C. Cui, W. Wang, M. Zhang, G. Chen, Z. Luo, and B. C. Ooi, “Alphaevolve: A learning framework to discover novel alphas in quantitative investment,” in *Proceedings of the 2021 International conference on management of data*, 2021, pp. 2208–2216.
- [18] T. Li, Z. Liu, Y. Shen, X. Wang, H. Chen, and S. Huang, “MASTER: Market-guided stock transformer for stock price forecasting,” in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 162–170.
- [19] Z. Li, R. Song, C. Sun, W. Xu, Z. Yu, and J.-R. Wen, “Can large language models mine interpretable financial factors more effectively? a neural-symbolic factor mining agent model,” in *Findings of the Association for Computational Linguistics ACL 2024*, 2024, pp. 3891–3902.
- [20] Y. Shi, Y. Duan, and J. Li, “Navigating the alpha jungle: An llm-powered mcts framework for formulaic factor mining,” *arXiv preprint arXiv:2505.11122*, 2025.
- [21] L. Kocsis and C. Szepesvári, “Bandit based monte-carlo planning,” in *European conference on machine learning*. Springer, 2006, pp. 282–293.
- [22] H. Lin, D. Zhou, W. Liu, and J. Bian, “Learning multiple stock trading patterns with temporal routing adaptor and optimal transport,” in *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*, 2021, pp. 1017–1026.